

Austin Crime Data Lakehouse

1. Project overview

The Austin Crime Data Lakehouse project will build a modern, reproducible data platform that ingests public crime-report data from the City of Austin Open Data Portal, preserves the original source data, processes and governs it, and makes it available for analytics, reporting, and responsible machine-learning use cases.

The initial source will be the City of Austin Crime Reports dataset, identified in the Socrata platform as `fdj4-gpfu`. The platform exposes the dataset through a programmable API that can be queried incrementally.

The project is primarily educational, but it will follow realistic data-engineering practices:

- Separation of storage, processing, orchestration, governance, and serving
 - Incremental and repeatable ingestion
 - Schema-drift management
 - Source-to-target reconciliation
 - Operational monitoring
 - Data quality testing
 - CI/CD and containerization
 - Clear, maintainable Python and SQL
 - Analytics and machine-learning support
-

2. Business objective

Objective statement

Create a reliable and auditable data platform that converts public City of Austin crime-report data into trusted analytical datasets for reporting, trend analysis, forecasting, and operational monitoring.

The platform should enable users to answer questions such as:

- How has reported crime volume changed over time?
 - Which offense categories occur most frequently?
 - How do reported incidents differ by district, sector, or location type?
 - How long does it take for incidents to be reported or cleared?
 - Which records were added, changed, archived, or removed by the source?
 - Are there unusual increases or decreases in reported incidents?
 - Can future weekly or monthly reporting volume be forecast?
-

3. Business problem statement

The City of Austin provides crime-report data through a third-party Socrata API. However, querying the API directly is not sufficient for reliable analytics because the source is outside our control.

The source may experience:

- Schema changes
- New or removed columns
- Field type changes
- Late-arriving records
- Updated historical records
- Deleted or archived records
- Duplicate records
- API downtime
- Paging interruptions
- Rate limits
- Incomplete extraction runs
- Full dataset replacements

Without a managed data pipeline, dashboards and analytical models could contain missing, duplicated, outdated, or inconsistent information.

The proposed lakehouse will preserve the source data, maintain historical changes, reconcile source and target records, and expose governed analytical tables.

4. Proposed modern architecture

flowchart LR

```
subgraph SOURCE["Source systems"]
    API["City of Austin<br/>Socrata API"]
end

subgraph CONTROL["Control and orchestration plane"]
    AF["Apache Airflow"]
    OPS["PostgreSQL<br/>Operational metadata"]
    ALERT["Logging and alerts"]
end

subgraph INGEST["Ingestion layer"]
    PY["Python Socrata client<br/>Pagination, retries, watermarks"]
end
```

```

subgraph STORAGE["Object storage layer"]
    RAW["Raw zone<br/>Immutable JSON"]
    QUAR["Quarantine zone<br/>Invalid or incompatible records"]
end

subgraph PROCESS["Processing layer"]
    SPARK["Apache Spark<br/>Validation, deduplication,<br/>MERGE and
enrichment"]
end

subgraph LAKEHOUSE["Lakehouse storage"]
    BRONZE["Bronze Iceberg tables<br/>Source-aligned"]
    SILVER["Silver Iceberg tables<br/>Cleaned and conformed"]
    GOLD["Gold Iceberg tables<br/>Analytics and ML features"]
end

subgraph CATALOG["Metadata and catalog layer"]
    POLARIS["Apache Polaris<br/>Iceberg REST catalog"]
end

subgraph SQL["SQL transformation and serving"]
    TRINO["Trino<br/>Distributed SQL engine"]
    DBT["dbt-trino<br/>Models, tests and documentation"]
end

subgraph CONSUMERS["Consumption layer"]
    BI["Power BI / Superset<br/>Dashboards and reporting"]
    NOTEBOOK["Python notebooks<br/>Exploratory analytics"]
    ML["MLflow + Python/Spark<br/>Forecasting and ML"]
end

API --> PY
AF --> PY
AF --> SPARK
AF --> DBT

PY --> RAW
PY --> OPS

RAW --> SPARK
SPARK --> BRONZE
SPARK --> SILVER
SPARK --> QUAR

POLARIS --- BRONZE
POLARIS --- SILVER
POLARIS --- GOLD

```

```
SILVER --> TRINO
TRINO --> DBT
DBT --> GOLD

GOLD --> TRINO
TRINO --> BI
TRINO --> NOTEBOOK
GOLD --> ML

AF --> OPS
SPARK --> OPS
DBT --> OPS
OPS --> ALERT
```

5. Architecture summary

Source layer

Tool: Socrata API

The City of Austin Open Data Portal is the external source system. Because the project does not control the source, the pipeline must assume that records and schemas may change.

The ingestion process should request both business columns and useful Socrata system fields, such as the source record ID and update timestamp.

Orchestration layer

Tool: Apache Airflow

Airflow will schedule and coordinate the pipeline but will not contain all business logic.

Airflow is designed for developing, scheduling, and monitoring batch-oriented workflows, which makes it appropriate for the planned incremental and reconciliation jobs.

Airflow will:

- Schedule ingestion
- Start Spark processing
- Run dbt models and tests
- Apply retry policies
- Record execution states

- Coordinate dependencies
- Trigger alerts
- Manage backfills
- Run reconciliation jobs

The Airflow DAG files should remain small. They should call reusable Python modules, Spark jobs, and dbt commands.

Ingestion layer

Tool: Python

A simple Python application will:

- Connect to the Socrata API
- Build API queries
- Apply incremental filters
- Retrieve paginated responses
- Handle retryable HTTP failures
- Preserve response metadata
- Write raw JSON
- Calculate record hashes
- Create batch manifests
- Record operational statistics

Airflow orchestrates this application, but the ingestion code should also be executable and testable without Airflow.

Object-storage layer

Tool: MinIO locally; Amazon S3 or equivalent later

The raw data will be stored in S3-compatible object storage.

MinIO provides an S3-compatible interface, allowing the local project to use common S3 libraries and later move to cloud object storage with relatively small configuration changes.

Suggested storage layout:

```
austin-crime-lakehouse/  
├─ landing/  
│   └─ crime_reports/  
│       └─ ingestion_date=YYYY-MM-DD/
```

```
|
|├─ bronze/
|│   └─ crime_reports/
|│
|├─ silver/
|│   └─ crime_reports/
|│
|├─ gold/
|│   ├── reporting/
|│   └─ ml_features/
|│
|├─ quarantine/
|│   └─ crime_reports/
|│
|└─ manifests/
|   └─ crime_reports/
```

Raw files should be immutable. Corrected data should be written as a new batch rather than modifying the original API response.

Lakehouse table format

Tool: Apache Iceberg

Iceberg will provide the table-management capabilities that distinguish the project from a basic data lake.

Iceberg is an open table format designed for analytical datasets and supports engines including Spark, Trino, and Flink.

It will provide:

- Table snapshots
- Schema evolution
- Partition evolution
- Transactional table commits
- Historical versions
- Time-travel queries
- Insert, update, delete, and merge operations
- Multi-engine access

The Iceberg tables will become the primary analytical data store. PostgreSQL will not need to hold a second complete copy of all analytical records.

Processing layer

Tool: Apache Spark

Spark will perform the heavier data-engineering operations.

The official Iceberg documentation describes Spark as its most feature-rich compute engine and recommends it as a starting point for learning Iceberg.

Spark responsibilities will include:

- Reading raw JSON
- Flattening the API payload
- Applying controlled data types
- Identifying schema changes
- Separating invalid records
- Standardizing timestamps and field names
- Deduplicating records
- Calculating source hashes
- Performing Iceberg `MERGE` operations
- Maintaining current and historical records
- Compacting small files
- Writing Bronze and Silver tables

Spark should handle technical data preparation. It should not contain reporting definitions that are better expressed in dbt.

Catalog and technical metadata layer

Tool: Apache Polaris

Polaris will act as the shared Iceberg catalog.

It implements the Iceberg REST catalog interface and supports interoperability among multiple engines, including Spark, Trino, and Flink.

Polaris will allow both Spark and Trino to locate and operate on tables such as:

```
austin_lakehouse.bronze.crime_reports  
austin_lakehouse.silver.crime_reports_current  
austin_lakehouse.silver.crime_reports_history  
austin_lakehouse.gold.fct_crime_incident
```

For the earliest implementation, a simpler JDBC-based Iceberg catalog can be used. Polaris can be introduced after the first Spark and Iceberg pipeline works.

SQL query layer

Tool: Trino

Trino will provide the main SQL interface over the lakehouse.

Its Iceberg connector supports querying data stored in Iceberg tables.

Trino will support:

- Interactive SQL
- dbt transformations
- BI connections
- Notebook queries
- Cross-source queries
- Analyst exploration
- Views and reporting tables

The separation is:

Spark = batch processing and table maintenance

Trino = interactive SQL and analytical serving

Flink is not required in the first project version because the Socrata source is batch-oriented rather than a continuous event stream.

Transformation layer

Tool: dbt with `dbt-trino`

dbt will connect to Trino and create SQL-based analytical models over the Iceberg tables. The official dbt documentation provides a `dbt-trino` adapter for connecting dbt to Trino.

dbt responsibilities will include:

- Staging models
- Business transformations
- Fact and dimension tables
- Aggregated reporting models

- SQL-based data-quality tests
- Model documentation
- Dependency lineage
- Incremental analytical models

Suggested layers:

```
Bronze Iceberg
  ↓
Silver Iceberg
  ↓
dbt staging
  ↓
dbt intermediate
  ↓
dbt marts
  ↓
Gold Iceberg
```

Suggested models:

```
models/
├─ staging/
│   └─ stg_crime_reports.sql
│
├─ intermediate/
│   ├── int_crime_reports_cleaned.sql
│   ├── int_crime_reports_current.sql
│   └─ int_crime_reports_history.sql
│
├─ dimensions/
│   ├── dim_date.sql
│   ├── dim_offense.sql
│   └─ dim_location.sql
│
├─ facts/
│   └─ fct_crime_incident.sql
│
└─ marts/
    ├── mart_daily_crime_summary.sql
    ├── mart_crime_by_district.sql
    ├── mart_clearance_rates.sql
    └─ mart_monthly_crime_trends.sql
```

Operational metadata layer

Tool: PostgreSQL

PostgreSQL will store operational and control-plane data rather than the complete analytical dataset.

It should contain separate logical databases or schemas:

```
PostgreSQL
├─ airflow_metadata
├─ ops_control
└─ optional_application_metadata
```

The `ops_control` database should include:

- Pipeline runs
- Task runs
- API request statistics
- Watermarks
- Extraction batches
- Raw-file manifests
- Record counts
- Schema observations
- Reconciliation results
- Data-quality results
- Failure information
- Retry information
- Alerts and manual resolutions

Airflow's internal metadata should remain separate from the custom operational metadata.

BI and reporting layer

Tools: Power BI or Apache Superset

BI tools will connect to Trino rather than directly to raw files.

Initial dashboards may include:

- Total reported incidents
- Reports by day, week, month, and year
- Incidents by council district
- Incidents by sector
- Incidents by offense category

- Family-violence trends
 - Clearance-rate trends
 - Average reporting delay
 - Average clearance delay
 - Data freshness
 - Pipeline run health
 - Reconciliation differences
-

Machine-learning layer

Tools: Python, Spark ML, and MLflow

The first ML use cases should use aggregated data rather than person-level predictions.

Potential use cases include:

- Weekly incident-volume forecasting
- Monthly incident-volume forecasting
- Workload forecasting by district
- Anomaly detection in reporting volume
- Source-data anomaly detection
- Pipeline failure prediction
- Data-quality anomaly detection

Suggested first feature table:

```
gold.ml_features_weekly_crime_volume
```

Example features:

```
week_start
district
offense_category
incident_count
previous_week_count
rolling_4_week_average
rolling_12_week_average
month
quarter
year
target_next_week_count
```

6. Schema-management strategy

The platform should not be completely schemaless.

Instead, it should become progressively more governed as data moves through the architecture.

Layer	Schema approach
Raw	Accept and preserve valid JSON
Bronze	Source-aligned, flexible schema
Silver	Explicit typed schema
Gold	Stable business schema
ML features	Versioned feature contract

Every raw record should contain an ingestion envelope:

```
{
  "metadata": {
    "dataset_id": "fdj4-gpfu",
    "source_record_id": "source-id",
    "source_updated_at": "2026-07-17T12:00:00Z",
    "ingested_at": "2026-07-17T12:10:00Z",
    "pipeline_run_id": "run-uuid",
    "batch_id": "batch-uuid",
    "payload_hash": "sha256-hash"
  },
  "payload": {
    "source_fields": "preserved here"
  }
}
```

Schema changes should be classified as:

```
ADDED_COLUMN
REMOVED_COLUMN
TYPE_CHANGED
FIELD_RENAMED
NULLABILITY_CHANGED
UNEXPECTED_VALUE
MALFORMED_RECORD
```

A new optional field should not stop raw ingestion. A breaking field-type change should preserve the source record, quarantine the typed version, and create an operational alert.

7. Pipeline 1: Incremental ingestion DAG

DAG name

```
crime_reports_incremental
```

Purpose

Retrieve records that were created or updated after the last successful extraction and load them into the lakehouse.

Process

1. Create pipeline-run record
2. Set status to RUNNING
3. Read the last successful watermark
4. Establish a fixed extraction window
5. Retrieve source metadata
6. Extract changed records from Socrata
7. Write immutable JSON files
8. Validate extraction manifest
9. Run Spark Bronze processing
10. Run Spark Silver merge
11. Run dbt models
12. Run dbt and data-quality tests
13. Validate source-to-target counts
14. Update the watermark
15. Set status to SUCCESS

The watermark must only be updated after all required processing and validation steps succeed.

An overlap period should be applied to the incremental query to protect against late-arriving updates:

```
effective_start =  
    last_successful_watermark - overlap_period
```

Duplicate records created by the overlap will be removed downstream using the source ID, update timestamp, and payload hash.

8. Pipeline 2: Reconciliation DAG

DAG name

```
crime_reports_reconciliation
```

Purpose

Compare the source system with lakehouse records and identify records that were missed, updated, duplicated, archived, or removed.

Process

1. Create reconciliation-run record
2. Retrieve source IDs and source hashes
3. Retrieve active lakehouse IDs and hashes
4. Compare the source and lakehouse
5. Classify all differences
6. Write reconciliation results
7. Automatically repair safe differences
8. Quarantine uncertain differences
9. Mark confirmed source deletions as inactive
10. Rebuild affected dbt models
11. Validate final counts
12. Complete the reconciliation run

Possible reconciliation results:

```
MATCHED  
MISSING_FROM_LAKEHOUSE  
MISSING_FROM_GOLD  
SOURCE_UPDATED  
SOURCE_DELETED  
HASH_MISMATCH  
DUPLICATE_SOURCE_RECORD  
DUPLICATE_TARGET_RECORD
```

Source deletions should not immediately remove historical data.

Instead:

```
is_active = false
source_deleted_at = reconciliation_timestamp
valid_to = reconciliation_timestamp
```

A record can be required to remain missing for two consecutive reconciliation runs before it is treated as a confirmed deletion.

9. Operational status model

The pipeline-run status should be separate from the failure reason.

Run statuses

```
PENDING
RUNNING
RETRYING
SUCCESS
SUCCESS_WITH_WARNINGS
FAILED
CANCELLED
MANUAL_INTERVENTION_REQUIRED
```

Failure categories

```
API_CONNECTION_ERROR
API_AUTHENTICATION_ERROR
API_RATE_LIMIT
INVALID_API_RESPONSE
SCHEMA_CHANGE
STORAGE_WRITE_ERROR
SPARK_PROCESSING_ERROR
CATALOG_COMMIT_ERROR
TRINO_QUERY_ERROR
DBT_MODEL_FAILURE
DBT_TEST_FAILURE
DATA_QUALITY_FAILURE
RECONCILIATION_FAILURE
MAX_RETRIES_EXCEEDED
UNKNOWN_ERROR
```

Example terminal state:

```
status = MANUAL_INTERVENTION_REQUIRED
failure_category = MAX_RETRIES_EXCEEDED
```

A successful HTTP response should not automatically mean the pipeline succeeded.

Pipeline success requires:

```
API extraction completed
AND all expected pages were received
AND raw data was written
AND manifests were validated
AND Spark committed the Iceberg tables
AND dbt models completed
AND required tests passed
AND the watermark was updated
```

10. Proposed implementation steps

Phase 1: API exploration

Build a notebook or basic Python script that:

- Requests 10–100 records
- Inspects response structure
- Identifies source IDs
- Identifies update timestamps
- Reviews null values
- Reviews data types
- Saves one JSON file locally

Deliverable:

```
notebooks/01_socrata_api_exploration.ipynb
```

Phase 2: Simple Python ingestion

Create a small Python application that:

- Accepts configuration

- Calls the API
- Handles pagination
- Writes newline-delimited JSON
- Creates a batch manifest
- Uses structured logging
- Includes simple error handling

Do not introduce Airflow yet.

Deliverable:

```
src/ingestion/  
├─ socrata_client.py  
├─ extractor.py  
├─ raw_writer.py  
└─ models.py
```

Phase 3: Object storage

Deploy MinIO with Docker Compose.

Create buckets or prefixes for:

- Raw
- Bronze
- Silver
- Gold
- Quarantine
- Manifests

Update the Python extractor to write to object storage.

Phase 4: Spark processing

Create two independent Spark jobs:

```
raw_to_bronze.py  
bronze_to_silver.py
```

Implement:

- JSON parsing

- Metadata-envelope extraction
 - Type conversion
 - Deduplication
 - Invalid-record handling
 - Record hashing
 - Current and history tables
-

Phase 5: Apache Iceberg

Configure Spark to write Iceberg tables.

Test:

- Table creation
- Inserts
- Updates
- Merges
- Schema evolution
- Snapshots
- Time travel

Iceberg-aware engines read table schemas from Iceberg metadata, allowing multiple compatible engines to work with the same tables.

Phase 6: Trino

Deploy a Trino coordinator and initially one worker.

Configure:

- Iceberg connector
- Object-storage credentials
- Iceberg catalog
- PostgreSQL connector for operational data

Validate basic SQL queries against Bronze and Silver tables.

Phase 7: dbt

Install and configure `dbt-trino`.

Create:

- One staging model
- One intermediate model
- One fact table
- Two reporting marts
- Basic schema tests
- dbt documentation

dbt supports table, view, incremental, ephemeral, and materialized-view strategies, depending on the adapter and target capabilities.

Phase 8: Operational metadata

Create the `ops_control` PostgreSQL schema.

Initial tables:

```
pipeline_run
pipeline_task_run
source_watermark
extract_batch
raw_file_manifest
schema_observation
reconciliation_run
reconciliation_result
data_quality_result
```

Update the independent Python and Spark jobs to write operational results.

Phase 9: Airflow orchestration

Only after the scripts work independently, create:

```
crime_reports_incremental_dag.py
crime_reports_reconciliation_dag.py
```

The DAGs should call the existing Python, Spark, and dbt components instead of reimplementing their logic.

Phase 10: BI dashboard

Connect Power BI or Superset to Trino.

Build:

- Data overview page
 - Geographic or district trends
 - Offense trends
 - Clearance analysis
 - Pipeline monitoring page
-

Phase 11: Machine learning

Create one baseline forecasting use case.

Recommended first model:

Forecast weekly reported-incident volume by district

Compare the model against a simple baseline such as:

Next week prediction =
previous four-week average

Record:

- Feature-table version
 - Training dates
 - Model parameters
 - Evaluation metrics
 - Model artifact
 - Prediction results
-

Phase 12: CI/CD

Use GitHub Actions for:

- Python formatting
- Linting
- Unit tests

- Mocked API tests
 - Spark transformation tests
 - DAG import tests
 - dbt compilation
 - dbt tests
 - Docker image builds
 - Integration testing
-

11. Recommended first release

The first end-to-end release should contain:

```
One Socrata dataset
One Python extractor
One MinIO raw bucket
One Bronze Iceberg table
One Silver Iceberg current table
One Silver history table
One Trino catalog
One dbt staging model
One dbt fact model
Two reporting marts
One incremental Airflow DAG
One reconciliation Airflow DAG
One operational PostgreSQL schema
One BI dashboard
```

Do not include Flink, Kafka, Kubernetes, or several datasets in the first release.

12. Future architecture additions

After the batch architecture is stable, future phases may add:

Streaming

```
Kafka → Flink → Iceberg
```

This would be justified only when a continuous event source becomes available.

Cloud deployment

MinIO → Amazon S3
Local Spark → managed Spark
Local PostgreSQL → managed PostgreSQL
Docker Compose → Kubernetes or cloud services

Governance and observability

Possible later additions:

- OpenMetadata
- DataHub
- OpenLineage
- Marquez
- Great Expectations
- Soda
- Prometheus
- Grafana

These should not be introduced before the fundamental ingestion and transformation pipeline is working.

13. Final proposed stack

Layer	Recommended tool
Source	City of Austin Socrata API
Ingestion	Python
Orchestration	Apache Airflow
Local object storage	MinIO
Cloud object storage	Amazon S3
Table format	Apache Iceberg
Batch processing	Apache Spark
Table catalog	Apache Polaris
SQL engine	Trino
SQL transformations	dbt-trino
Operational database	PostgreSQL

Layer	Recommended tool
BI	Power BI or Apache Superset
ML	Python, Spark ML, MLflow
Local deployment	Docker Compose
CI/CD	GitHub Actions

The central design is:

```

Airflow orchestrates.
Python extracts.
Object storage preserves.
Spark processes.
Iceberg manages tables.
Polaris catalogs.
Trino queries.
dbt defines business logic.
PostgreSQL tracks operations.
BI reports.
ML forecasts.

```